

 AVALIAÇÃO

Nome:					R.A.:		
Curso:	Ciência da Computação				Data:	____/____/2026	
Disciplina:	Estrutura de Dados - 4 período				Docente	Prof. Me. Karan Luciano	
Avaliação:	N1	N2	Sub	Exame	Nota:		

Antes de iniciar a avaliação, leia atentamente as orientações abaixo:

- 1 – Confira se este caderno contém todas as questões objetivas (múltipla escolha);
- 2 – **Não será permitido o uso do telefone celular ou qualquer outro dispositivo eletrônico. Os mesmos devem ser desligados e guardados ou entregues ao professor;**
- 3 – As questões objetivas contêm apenas uma resposta correta. A marcação de mais de uma resposta invalidará individualmente a questão;
- 4 – Transcreva as respostas das questões objetivas no Gabarito, sem rasuras;
- 5 – No gabarito das questões objetivas, marque somente aquelas correspondentes à sua avaliação, com um "X";
- 6 – Use caneta esferográfica transparente de tinta azul ou preta;
- 7 – **Não será permitido nenhum tipo de consulta;**
- 8 – Não utilize calculadora (exceto em caso de permissão pelo professor), não se comunique com os demais acadêmicos e não troque quaisquer materiais com estes;
- 9 – Não será permitido o uso de outros papéis para rascunho ou qualquer outra finalidade;
- 10 – **Será atribuída a nota zero ao aluno que utilizar de meio fraudulento durante a avaliação;**
- 11 – O tempo mínimo de permanência em sala de aula é de 30 minutos;
- 12 – Os últimos 03 alunos deverão permanecer em sala até a conclusão da avaliação por estes;
- 13 – Após concluir a avaliação, entregue o caderno de prova ao professor;
- 14 – Preencha o Gabarito com cuidado.

Desejamos uma excelente avaliação!

GABARITO – QUESTÕES OBJETIVAS

Marque apenas as questões correspondentes à sua avaliação. Rasuras invalidam a sua resposta.

1	A	B	C	D	E	6	A	B	C	D	E
2	A	B	C	D	E	7	A	B	C	D	E
3	A	B	C	D	E	8	A	B	C	D	E
4	A	B	C	D	E	9	A	B	C	D	E
5	A	B	C	D	E	10	A	B	C	D	E

QUESTÕES OBJETIVAS

QUESTÃO 01

Ponteiros e Gestão de Memória

Em linguagem C, a memória física (RAM) é organizada sequencialmente como uma vasta sequência de bytes numerados. Ao declararmos uma variável comum e um ponteiro associado a ela, como em `int x = 42; int *ptr = &x;`, qual é a finalidade precípua da variável `ptr` na arquitetura de execução?

- A) Guardar o endereço de memória física onde a variável referenciada está armazenada.
- B) Duplicar automaticamente o valor de `x` em uma área reservada de backup da memória.
- C) Executar a limpeza e liberação automática do espaço ocupado por `x` no término do bloco.
- D) Armazenar valores numéricos fracionários de ponto flutuante de dupla precisão.
- E) Converter o valor decimal de `x` em um tipo caractere imutável.

QUESTÃO 02

Operadores de Indireção

Considere a execução sequencial do seguinte trecho de código em linguagem C:

```
int numero = 10;
int *ptr = &numero;
*ptr = 25;
```

Após o processamento dessas três instruções, qual será o valor impresso pela instrução `printf("%d", numero);` e qual mecanismo fundamenta essa alteração?

- A) Imprime 10, pois ponteiros operam apenas sobre cópias locais isoladas em registradores.
- B) Imprime 25, pois o operador de desreferenciação (*) altera diretamente a posição de memória original de `numero`.
- C) Imprime o endereço hexadecimal da variável `numero` na pilha de execução (*Stack*).
- D) Imprime 0, pois a atribuição via ponteiro reinicializa o conteúdo da variável apontada.
- E) Ocorre um erro de violação de acesso (*Segmentation Fault*) por atribuição ilegal de tipo inteiro a ponteiro.

QUESTÃO 03

Aritmética de Ponteiros

Em uma arquitetura computacional padrão onde o tipo de dado `int` ocupa exatamente 4 bytes de memória, um desenvolvedor declara o vetor `int v[5] = {10, 20, 30, 40, 50};`. Sabendo que o início do vetor reside no endereço hexadecimal `0x1000`, a expressão `*(v + 3)` irá acessar:

- A) O endereço físico `0x1003`, retornando o valor 30.
- B) O endereço físico `0x100C` (deslocamento de 3×4 bytes), recuperando o valor 40.
- C) O elemento de índice 2 do vetor, resultando no valor numérico 30.
- D) Um erro em tempo de compilação, pois identificadores de vetores não admitem operações aritméticas.
- E) O endereço base `0x1000`, somando 3 unidades diretamente ao valor 10 e resultando em 13.

QUESTÃO 04

Alocação Dinâmica

Ao construir estruturas de dados que necessitam crescer ou encolher durante o tempo de execução do programa (como listas encadeadas e árvores), a alocação estática na pilha (*Stack*) torna-se insuficiente. Para requisitar blocos contíguos de memória sob demanda diretamente da *Heap*, qual função padrão da biblioteca `<stdlib.h>` deve ser invocada?

- A) `malloc()`, especificando a quantidade necessária de bytes via operador `sizeof`.
- B) `sizeof()`, encarregada de requisitar e inicializar blocos na memória dinâmica.
- C) `printf()`, que aloca buffers dinâmicos de saída padrão.
- D) `system()`, que solicita ao sistema operacional a criação de uma nova partição de memória.
- E) `realloc()`, a única permitida para criar ponteiros dinâmicos a partir do zero.

QUESTÃO 05

Ciclo de Vida da Memória

Durante a execução contínua de um serviço de retaguarda (*backend*) desenvolvido em C, observou-se um consumo progressivo da memória RAM da máquina servidora, culminando na interrupção abrupta do processo pelo sistema operacional (*Out of Memory Killer*). A auditoria identificou blocos instanciados repetidamente com `malloc()` sem a devida chamada a `free()`. Esse cenário caracteriza formalmente:

- A) Falha de segmentação (*Segmentation Fault*) induzida pela destruição da tabela de símbolos.
- B) Um vazamento de memória (*Memory Leak*), pois a memória na *Heap* permanece reservada e inacessível pelo encerramento dos ponteiros locais.
- C) Atuação correta do Coletor de Lixo (*Garbage Collector*) nativo do C, que compacta blocos ociosos.
- D) Transbordamento de pilha (*Stack Overflow*) decorrente de chamadas recursivas infinitas.
- E) Corrupção instantânea dos registradores da CPU por ausência do comando `return`.

QUESTÃO 06*Registros e Nós Dinâmicos*

Considere a declaração de um nó típico de estrutura dinâmica em C:

```
typedef struct No {  
    int valor;  
    struct No *proximo;  
} No;  
No *ponteiro_no = (No*) malloc(sizeof(No));
```

Para atribuir o número 100 ao campo `valor` através do ponteiro `ponteiro_no`, qual sintaxe em conformidade com o padrão ANSI C deve ser adotada?

- A) `ponteiro_no.valor = 100;`
- B) `ponteiro_no->valor = 100;` ou de forma equivalente `(*ponteiro_no).valor = 100;`
- C) `ponteiro_no::valor = 100;`
- D) `&ponteiro_no->valor = 100;`
- E) `*ponteiro_no->valor = 100;`

QUESTÃO 07*Algoritmos de Busca*

A Busca Binária é um algoritmo de alta performance amplamente empregado na recuperação de informações. Contudo, diferentemente da Busca Sequencial (Linear), ela impõe uma pré-condição mandatória sobre a estrutura de dados para garantir sua corretude. Essa pré-condição consiste em:

- A) A coleção de dados deve possuir obrigatoriamente um número par de elementos.
- B) Os elementos devem estar previamente organizados em ordem monotônica (estritamente ordenados de forma crescente ou decrescente).
- C) A estrutura deve ser obrigatoriamente alocada de forma dinâmica por meio de nós e ponteiros.
- D) O vetor não pode conter elementos repetidos ou valores numéricos nulos.
- E) A busca precisa ser executada obrigatoriamente por múltiplos núcleos de processamento em paralelo.

QUESTÃO 08*Análise de Complexidade*

Em uma base cadastral contendo aproximadamente 1.000.000 (um milhão) de registros devidamente ordenados, um algoritmo de busca sequencial pode requerer, no pior caso, até 1.000.000 de comparações para encontrar uma chave ou constatar sua ausência ($O(n)$). Em contrapartida, a Busca Binária resolve a mesma tarefa com no máximo 20 comparações. Essa superioridade deve-se ao fato de que:

- A) A Busca Binária possui complexidade de tempo logarítmica ($O(\log_2 n)$), reduzindo pela metade o espaço de busca a cada iteração.
- B) A Busca Binária acessa diretamente a posição exata por cálculo de dispersão (*hash*) em tempo constante $O(1)$.
- C) O algoritmo transforma o vetor linear em uma matriz multidimensional para executar buscas diagonais.
- D) Ela realiza comparações simultâneas em todos os índices utilizando recursos da Unidade de Ponto Flutuante (FPU).
- E) A Busca Binária ignora os valores intermediários e compara apenas o elemento inicial e o final.

QUESTÃO 09*Algoritmos de Ordenação Padrões*

O *Bubble Sort* (Método da Bolha) é um dos algoritmos de ordenação elementares mais conhecidos. Sobre sua mecânica de funcionamento e comportamento assintótico, assinale a proposição correta:

- A) Opera selecionando repetidamente o menor item da partição não ordenada e posicionando-o no início do vetor.
- B) Percorre a coleção comparando pares de elementos adjacentes e efetuando trocas quando fora de ordem, apresentando complexidade de tempo de pior caso $O(n^2)$.
- C) Divide iterativamente o conjunto ao meio até atingir partições atômicas unitárias, garantindo tempo $O(n \log n)$ em todos os casos.
- D) Consegue ordenar vetores estritamente decrescentes efetuando apenas uma única passada linear ($O(n)$).
- E) Necessita alocar um vetor auxiliar secundário com o mesmo tamanho da entrada para consolidar os dados.

QUESTÃO 10*Algoritmos Avançados de Ordenação*

O algoritmo *Merge Sort* destaca-se pela estabilidade e previsibilidade de desempenho, garantindo complexidade assintótica $O(n \log_2 n)$ em todos os seus cenários (melhor, médio e pior caso). O princípio metodológico que rege a estrutura do Merge Sort baseia-se em:

- A) Paradigma Guloso (*Greedy*), que seleciona a menor decisão imediata em cada passo sem possibilidade de retrocesso.
- B) Divisão e Conquista, decompondo recursivamente o problema em subproblemas menores até casos triviais, combinando-os subsequentemente de forma ordenada (*intercalação*).
- C) Busca Exaustiva com Retrocesso (*Backtracking*), testando todas as permutações possíveis dos itens até encontrar a configuração ordenada.
- D) Programação Dinâmica com memoização estrita em matriz bidimensional de adjacências.

E) Trocas aleatórias estocásticas baseadas no método de Monte Carlo.

QUESTÕES DISCURSIVAS E ANÁLISE DE CÓDIGO

QUESTÃO 11 (ANÁLISE DE CÓDIGO)

Ponteiros, Passagem por Referência e Escopo

Um estudante de Ciência da Computação desenvolveu o programa abaixo em linguagem C com o intuito de implementar uma rotina de inversão de valores entre duas variáveis e manipulação vetorial via ponteiros:

```
#include <stdio.h>

void misterio(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp + 5;
}

int main() {
    int v[4] = {10, 20, 30, 40};
    int *p1 = &v[1];
    int *p2 = &v[3];

    misterio(p1, p2);

    for (int i = 0; i < 4; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");
    return 0;
}
```

Com base na execução estrita do código acima pelo compilador, responda fundamentadamente:

- a) (1,0 pt) Apresente a **saída exata** impressa no terminal pelo laço `for` contido na função `main` ao término da execução do programa.
- b) (1,0 pt) Explique tecnicamente por que os elementos do vetor `v` foram permanentemente modificados pela função `misterio`, diferenciando o comportamento observado de uma passagem tradicional de parâmetros **por valor**.

QUESTÃO 12 (ANÁLISE DE CÓDIGO)

Encadeamento Dinâmico e Ciclo de Vida na Heap

Considere a rotina de manipulação de nós dinâmicos em C apresentada a seguir:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int info;
    struct Node *next;
} Node;

int main() {
    Node *n1 = (Node*) malloc(sizeof(Node));
    Node *n2 = (Node*) malloc(sizeof(Node));

    n1->info = 10;
    n1->next = n2;

    n2->info = 20;
    n2->next = NULL;

    // Trecho sob auditoria
    Node *p = n1;
    while (p != NULL) {
        printf("[%d] -> ", p->info);
        p = p->next;
    }
    printf("NULL\n");

    free(n1);
    // Linha final do desenvolvedor:
    n2 = NULL;
    return 0;
}

```

Com base na gestão de memória RAM durante a execução desta rotina, responda:

- a) **(1,0 pt)** Qual é a saída impressa no console pela execução do laço `while` e como os ponteiros `n1`, `n2` e `next` conectam logicamente esses dois nós na memória?

- b) **(1,0 pt)** Aponte explicitamente a falha crítica de gerenciamento de memória cometida no encerramento da função (`free(n1); n2 = NULL;`). Identifique qual patologia de memória ocorre (*Memory Leak* ou *Dangling Pointer*), em qual bloco ela ocorre e apresente a correção adequada em código C.

QUESTÃO 13 (DISCURSIVA)*Paradigmas de Busca e Análise de Complexidade*

Uma empresa de logística monitora o rastreamento em tempo real de encomendas rodoviárias em uma tabela com 500.000 pacotes registrados. O arquiteto do sistema deve escolher entre a implementação de um algoritmo de **Busca Sequencial (Linear)** e um algoritmo de **Busca Binária** para atender consultas concorrentes de alta frequência efetuadas pelos clientes no aplicativo móvel.

- a) (1,0 pt) Preencha o quadro comparativo abaixo, indicando o pré-requisito de ordenação, a quantidade máxima de comparações no pior caso (expressa numericamente ou em ordem assintótica Big-O) e a principal vantagem prática de cada método:

Critério de Análise	Busca Sequencial (Linear)	Busca Binária
Pré-requisito dos dados		
Complexidade (Pior Caso)		
Cenário de uso recomendado		

- b) (1,0 pt) Suponha que a coleção receba dezenas de novas encomendas a cada minuto, inseridas de forma desordenada no fim do vetor. Discuta o impacto do custo de ordenação prévia do vetor sobre a viabilidade econômica e computacional do uso contínuo da Busca Binária nesse cenário dinâmico.

QUESTÃO 14 (DISCURSIVA)*Ordenação e Escalabilidade de Algoritmos*

Ao projetar o módulo de ordenação de relatórios gerenciais, uma equipe de desenvolvimento avalia a adoção do algoritmo **Bubble Sort** ou do algoritmo **Merge Sort**. Em testes preliminares em bancada com um vetor de apenas 20 elementos inteiros, ambos os métodos apresentaram tempo de execução imperceptível (frações de milissegundo). Contudo, ao submeter uma base de homologação contendo 1.000.000 (um milhão) de registros desordenados, o sistema congelou sob o Bubble Sort, enquanto o Merge Sort concluiu a ordenação em segundos.

- a) (1,0 pt) Utilizando os fundamentos matemáticos da **Notação Big-O**, explique por que ocorreu essa discrepância gritante de tempo no teste com 1.000.000 de registros, contrastando as complexidades de pior caso de ambos os algoritmos ($O(n^2)$ vs. $O(n \log_2 n)$).

- b) (1,0 pt) Embora o Merge Sort seja assintoticamente muito superior ao Bubble Sort em tempo de execução, ele apresenta uma desvantagem expressiva quanto ao **consumo de memória espacial** ($O(n)$ adicional). Explique tecnicamente por que o Merge Sort necessita dessa memória auxiliar e em que situações de sistemas embarcados de baixo custo um algoritmo elementar *in-place* pode ser preferível.